

Universidad Central «Marta Abreu» de Las Villas
Facultad de Matemática, Física y Computación
Carrera: Ciencias de la Computación — Asignatura: Compiladores

CurrencyScript

*Documento Técnico — Analizador Léxico, Sintáctico y Semántico
de un Lenguaje de Dominio Específico para Operaciones Monetarias*

Proyecto Final — Estudiante 2

Celin Álvarez Vega

Asignatura: Compiladores

Curso: Tercer año — Ciencias de la Computación

Fecha: Julio de 2026

1. Descripción del DSL

CurrencyScript es un lenguaje de dominio específico para operaciones financieras con distintas monedas. El usuario declara tasas de cambio, define variables con un tipo monetario concreto (USD, EUR, COP, MXN, BRL) y el analizador garantiza que las operaciones sean semánticamente coherentes: no es posible sumar dólares con euros sin una conversión explícita previa, y toda conversión requiere que exista una tasa —directa o encadenada a través de monedas intermedias— que la respalde.

El objetivo del analizador no es ejecutar las operaciones, sino leer programas escritos en CurrencyScript, validarlos en las tres fases clásicas del análisis (léxica, sintáctica y semántica) y reportar errores significativos con línea, columna y una descripción clara del problema.

2. Palabras reservadas

`moneda`, `tasa`, `convertir`, `en`, `var`, `imprimir`, `total`, `si`, `sino`. Las monedas predefinidas (USD, EUR, COP, MXN, BRL) se clasifican como tokens de tipo `CURRENCY`, no como palabras reservadas, para permitir extensibilidad futura del sistema de monedas sin tocar el lexer.

3. Tabla completa de tokens

Token	Expresión regular / literal	Descripción
MONEDA	<code>moneda</code>	Declaración de tipo moneda
TASA	<code>tasa</code>	Declaración de tasa de cambio
CONVERTIR	<code>convertir</code>	Conversión entre monedas
EN	<code>en</code>	Indicador de destino en conversión
VAR	<code>var</code>	Declaración de variable
IMPRIMIR	<code>imprimir</code>	Impresión de un valor o cadena
TOTAL	<code>total</code>	Suma de variables declaradas de una moneda
SI / SINO	<code>si / sino</code>	Estructura de control condicional
CURRENCY	<code>USD EUR COP MXN BRL</code>	Monedas predefinidas del sistema
IDENTIFIER	<code>[a-zA-Z_][a-zA-Z0-9_]*</code>	Nombres de variables y tasas
NUMBER	<code>[0-9]+(\.[0-9]+)?</code>	Literales enteros y decimales
STRING	<code>"..." con \n \t \</code>	Cadenas entre comillas dobles, usadas en <code>imprimir</code>
ASSIGN	<code>=</code>	Asignación / inicialización
PLUS / MINUS / STAR / SLASH	<code>+ - * /</code>	Operadores aritméticos
LT / GT / LE / GE / EQ / NEQ	<code>< > <= >= == !=</code>	Operadores relacionales (usados en <code>si</code>)
SEMICOLON / COLON	<code>; :</code>	Fin de sentencia / separador de tipo
LPAREN / RPAREN	<code>()</code>	Paréntesis
LBRACE / RBRACE	<code>{ }</code>	Delimitadores de bloque (<code>si / sino</code>)

Los comentarios de línea (`//`) y de bloque (`/* */`) se descartan en la fase léxica y no generan tokens.

4. Gramática formal en EBNF

```

programa      ::= sentencia* EOF

sentencia     ::= tasaDecl | varDecl | assignStmt | printStmt | totalStmt | ifStmt

tasaDecl     ::= 'tasa' IDENTIFIER '=' signoNum ';'
varDecl      ::= 'var' IDENTIFIER ':' CURRENCY '=' expresion ';'
assignStmt   ::= IDENTIFIER '=' expresion ';'
printStmt    ::= 'imprimir' ( STRING | expresion ) ';'
totalStmt    ::= 'total' CURRENCY ';'
ifStmt       ::= 'si' '(' condicion ')' '{' sentencia* '}'
              ( 'sino' '{' sentencia* '}' )?

condicion     ::= expresion ( ('<' | '>' | '<=' | '>=' | '==' | '!=') expresion )?

expresion     ::= convertExpr | addExpr
convertExpr   ::= 'convertir' addExpr 'en' CURRENCY
addExpr       ::= mulExpr ( ('+' | '-') mulExpr )*
mulExpr       ::= unario ( ('*' | '/') unario )*
unario        ::= '-' primaria | primaria
primaria      ::= NUMBER | IDENTIFIER | '(' expresion ')'

signoNum      ::= '-' NUMBER | NUMBER

CURRENCY      ::= 'USD' | 'EUR' | 'COP' | 'MXN' | 'BRL'
IDENTIFIER    ::= [a-zA-Z][a-zA-Z0-9_]*
NUMBER        ::= [0-9]+ ( '.' [0-9]+ )?
STRING        ::= '"' ( [^"\\] | '\\' [nt] ) * '"'
comentario    ::= '//' [^\n]* | '/*' .*? '*/' (* ignorados *)

```

La gramática implementa cinco niveles de precedencia: (1) unario, (2) multiplicación/división, (3) suma/resta, (4) conversión y comparación relacional a nivel de sentencia, (5) agrupación con paréntesis. El parser es LL(1) de descenso recursivo.

5. Reglas semánticas

5.1 Suma entre monedas distintas — error

No se puede sumar ni restar variables de tipos monetarios diferentes, ni tampoco mezclar una moneda con un número sin moneda en una suma o resta (sí se permite en multiplicación/división, ver 5.6).

```

var a : USD = 10;
var b : EUR = 5;
imprimir a + b; // Error: no se puede sumar 'USD' con 'EUR'

```

5.2 Tasa requerida antes de convertir (directa o transitiva)

La operación `convertir X en MONEDA` requiere que exista una tasa —directa o alcanzable mediante una cadena de tasas intermedias— entre el origen y el destino. El analizador construye un grafo no dirigido con todas las tasas declaradas y visibles en el punto del programa (cada `tasa ORIGEN_a_DESTINO` agrega una arista en ambos sentidos) y ejecuta una búsqueda en amplitud (BFS).

```

tasa USD_a_EUR = 0.92;
tasa EUR_a_COP = 4500;
var salario : USD = 2000;
imprimir convertir salario en COP;
// Válido: no existe tasa directa USD_a_COP, pero sí la cadena USD -> EUR -> COP.

```

5.3 Tasas positivas

El valor de una tasa de cambio debe ser estrictamente mayor que cero.

```

tasa USD_a_EUR = -0.5;
// Error: la tasa 'USD_a_EUR' debe ser estrictamente positiva

```

5.4 Reglas obligatorias de las Orientaciones Generales

- **Variable usada sin declarar:** se reporta al resolver un `Identifier` que no existe en el ámbito actual ni en ninguno de sus ámbitos padre.
- **Variable redeclarada en el mismo ámbito:** se reporta si el nombre ya existe en el *mismo* objeto `Ambito` (no en un ámbito padre, lo cual sí es válido: sombreado de variables).
- **Tipos incompatibles en operaciones:** cubre sumas/restas entre monedas distintas (5.1), multiplicaciones entre dos monedas (5.6) y comparaciones relacionales entre monedas distintas (5.7).

- **Uso antes de asignación:** ver justificación de diseño en 5.5.

5.5 Sobre "uso antes de asignación" en este dominio

En CurrencyScript, declarar una variable (`var`) y asignarle su valor inicial ocurren en la misma sentencia; no existe una forma de declarar sin inicializar. Por ello, "uso antes de asignación" se interpreta aquí como **uso antes del punto de declaración**: el análisis semántico recorre el programa de forma estrictamente secuencial y sin *hoisting*, de modo que referenciar una variable o una tasa antes de la línea donde se declara produce el mismo error que usar un identificador nunca declarado. Esta equivalencia se documenta explícitamente porque no es la única interpretación posible del requisito general, y otra ficha del proyecto (con variables declaradas sin inicializar) exigiría una implementación distinta.

5.6 Multiplicación y división con escalares

Multiplicar o dividir una moneda por un número puro es válido (y necesario para calcular descuentos, impuestos, etc.); el resultado conserva la moneda. Multiplicar o dividir dos monedas entre sí no tiene interpretación económica clara en este dominio y es error.

```
var precio : USD = 100;
imprimir precio * 2;      // válido, resultado en USD
imprimir precio / 4;      // válido, resultado en USD
var otro : EUR = 5;
imprimir precio * otro;    // Error: no se pueden multiplicar dos monedas entre sí
```

5.7 Comparaciones relacionales (condición del `si`)

Comparar una moneda contra un número puro es válido (el número se interpreta como un umbral en esa misma moneda: `si (salario > 1500)`). Comparar dos monedas distintas entre sí es error, igual que en la suma.

5.8 Tabla de símbolos con ámbitos anidados

Cada bloque `{ }` de un `si/sino` abre un nuevo ámbito, encadenado al ámbito que lo contiene. La resolución de un identificador busca primero en el ámbito actual y, si no lo encuentra, recorre la cadena de ámbitos padre hasta el ámbito global. Esto permite, por ejemplo, declarar dentro de un bloque `si` una variable temporal (un bono, un recargo) sin que su nombre choque con variables del mismo nombre en otro bloque hermano, y sin contaminar el ámbito global.

6. Decisiones de diseño

- **Herramienta elegida: JavaScript puro, sin generadores externos.** El DSL es pequeño (gramática LL(1)) y la implementación manual del léxico y del parser de descenso recursivo da control total sobre el manejo de errores posicionales y la recuperación en modo pánico, sin la curva de aprendizaje de una herramienta como ANTLR para un lenguaje de este tamaño.
- **Recuperación de errores:** modo pánico. Al detectar un error sintáctico, el parser avanza tokens hasta un punto de sincronización (`;`, `}`, o el inicio de una sentencia reconocible) y continúa, permitiendo reportar varios errores sintácticos en una sola pasada en lugar de detenerse en el primero.
- **Monedas como tokens separados de las palabras reservadas:** se clasifican como `CURRENCY` en vez de `KEYWORD` para poder añadir nuevas monedas sin modificar la lista de palabras reservadas del lenguaje.
- **Bidireccionalidad y transitividad de tasas:** la búsqueda de una tasa admite tanto el nombre en orden inverso como cadenas a través de monedas intermedias, ya que en la práctica una tasa conocida en una dirección —o a través de un puente común— es económicamente invertible/derivable.

7. Instrucciones para ejecutar

```
cd currencyscript/
node main.js

# Resultado esperado:
# prueba_valida.cs          -> OK
# prueba_error_sintactico.cs -> ERROR (fase sintactica)
# prueba_error_semantico.cs -> ERROR (fase semantica)
# prueba_extendida.cs       -> ERROR (fase semantica)
```

Los cuatro programas están en `pruebas/` como archivos `.cs` independientes.

